16-Point Radix-2 DIT FFT

Butterfly Datapath and Top-Level Interconnect — Design Report

1 The Discrete Fourier Transform and the Need for the FFT

The Discrete Fourier Transform (DFT) converts a sequence of N time-domain samples $x[n]$ into N frequency-domain components $X[k]$, revealing the frequency content of a signal. For a sequence of length N, it is defined as:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, W_N^{nk}, \qquad k = 0, 1, \ldots, N-1$$

where the *twiddle factor* is the complex exponential

$$W_N^{nk} = e^{-j\frac{2\pi}{N}nk} = \cos\left(\frac{2\pi nk}{N}\right) - j\sin\left(\frac{2\pi nk}{N}\right)$$

Computed directly, this equation requires N complex multiplications and $N-1$ complex additions for *every one* of the N outputs – an $O(N^2)$ operation. For $N = 16$ that is 256 complex multiplications, which is small, but the same direct approach at $N = 1024$ or $N = 4096$ (typical in audio/RF systems) becomes computationally prohibitive for real-time hardware.

The Fast Fourier Transform (FFT), specifically the Cooley–Tukey radix-2 algorithm used in this design, exploits the periodicity and symmetry of W_N to recursively decompose an N-point DFT into two $(N/2)$-point DFTs, then four $(N/4)$-point DFTs, and so on down to trivial 2-point DFTs. This reduces the complexity from

$$O(N^2) \;\longrightarrow\; O(N \log_2 N)$$

For $N = 16$, $\log_2(16) = 4$ stages of computation are required, each stage containing $N/2 = 8$ basic *butterfly* operations — 32 butterflies total instead of 256 raw multiplications, which is exactly the structure implemented in this design.

1.1 Decimation-in-Time (DIT)

This design uses the Decimation-in-Time variant of the algorithm. The input sequence is first split into even- and odd-indexed sub-sequences,

$$X[k] = \underbrace{\sum_{r=0}^{N/2-1} x[2r]\, W_{N/2}^{rk}}_{\text{even-indexed DFT}} + W_N^k \underbrace{\sum_{r=0}^{N/2-1} x[2r+1]\, W_{N/2}^{rk}}_{\text{odd-indexed DFT}}$$

and this splitting is applied recursively. The consequence of this repeated even/odd splitting is that the inputs to the very first computation stage must be supplied in **bit-reversed order** rather than natural order, while the outputs are produced in **natural order** after the final stage. This is the classic "bit-reversed in, natural order out" property of DIT FFTs, and it directly shapes how the input pairs are wired into Stage 1 of the top module (Section 3).

2 The Butterfly Operation

The butterfly is the atomic computational unit of the FFT. Each butterfly accepts a pair of complex inputs (A, B) and one complex twiddle factor W, and produces a new complex pair (A', B') using *one* complex multiplication and *two* complex additions:

$$A' = A + W \cdot B \qquad B' = A - W \cdot B$$

Writing the complex multiplication out in real/imaginary components, with $A = a_r + ja_i$, $B = b_r + jb_i$, $W = w_r + jw_i$:

$$\mathrm{Re}(W \cdot B) = b_r w_r - b_i w_i \qquad \mathrm{Im}(W \cdot B) = b_r w_i + b_i w_r$$

$$A'_r = a_r + \mathrm{Re}(W \cdot B) \quad A'_i = a_i + \mathrm{Im}(W \cdot B)$$

$$B'_r = a_r - \mathrm{Re}(W \cdot B) \quad B'_i = a_i - \mathrm{Im}(W \cdot B)$$

This is precisely what `butterfly.v` implements: one complex multiply (4 real multiplies, 2 real add/subtracts) followed by a complex add and a complex subtract, reusing the same product $W \cdot B$ for both outputs. This reuse — computing $W \cdot B$ once and combining it with A both ways — is what gives the FFT its efficiency over the brute-force DFT.

2.1 Fixed-Point Arithmetic (Q1.14) and Scaling

All signals are 16-bit signed fixed point in **Q1.14** format: 1 sign/integer bit and 14 fractional bits, representing real values in the range $[-1.0, +1.0)$. A stored code c (as a signed integer) maps to a real value by

$$v = \frac{c}{2^{14}}$$

The twiddle factors (cosines/sines, always ≤ 1 in magnitude) fit this format exactly, and 1.0 itself is represented as the boundary code `0x4000`.

- **Multiplication:** multiplying two Q1.14 numbers produces a 32-bit Q2.28 result (`bw_r_full`, `bw_i_full`):

  $$\text{Q1.14} \times \text{Q1.14} \;\rightarrow\; \text{Q2.28}, \qquad 2^{-14} \cdot 2^{-14} = 2^{-28}$$

  The product is re-normalized back to Q1.14 by extracting bits `[29:14]`, which discards the extra integer growth bit and the low-order fractional bits below Q1.14 precision — equivalent to a right shift of 14.

- **Addition/Subtraction:** a_r and bw_r are each valid Q1.14 numbers in $[-1, 1)$, so their sum or difference can momentarily require 2 integer bits, i.e. range $[-2, 2)$:

  $$a_r,\, bw_r \in [-1, 1) \implies a_r \pm bw_r \in [-2, 2)$$

  The code sign-extends both operands to 17 bits before adding/subtracting (`sum_r`, `diff_r`) to guarantee this growth is captured without overflow or wraparound.

- **Re-scaling:** the 17-bit sum/difference is arithmetically shifted right by 1 (`>>>1`) and truncated back to 16 bits:

  $$\text{out} = \left\lfloor \frac{a_r \pm bw_r}{2} \right\rfloor$$

  This divides the result by 2, keeping every stage's output magnitude safely inside the representable range even in the worst case where all inputs are near full scale. The trade-off is a well-understood gain factor of $\frac{1}{2}$ introduced at *every* one of the four stages, i.e. an overall gain of

  $$\left(\frac{1}{2}\right)^4 = \frac{1}{16}$$

  across the whole 16-point transform, which is simply accounted for by the user of the FFT output rather than by extra hardware.

2.2 Pipelining

The four combinational outputs (`ar_out`, `ai_out`, `br_out`, `bi_out`) are captured into registers on the rising edge of `clk`, gated by `en` and cleared asynchronously by active-low `rst_n`. This turns every butterfly into a one-cycle pipeline stage: a fresh input pair is latched out one clock after it is presented, which is what allows the four FFT stages in the top module to be chained directly together as a 4-deep pipeline (Section 3.3).

3 Top-Level Module: `FFT_16Point`

The top module wires together 32 instances of the butterfly module

$$\frac{N}{2} \times \log_2 N \;=\; 8 \times 4 \;=\; 32 \text{ butterflies}$$

(8 per stage × 4 stages) into the full 16-point radix-2 DIT pipeline. Each stage halves the "distance" H between the two elements a butterfly operates on relative to the recursive DFT decomposition, while doubling the number of distinct twiddle angles used, exactly mirroring the Cooley–Tukey recursion described in Section 1.

3.1 Bit-Reversed Input Ordering

Because this is a Decimation-in-Time FFT, Stage 1 cannot simply pair adjacent samples (x_0 with x_1, x_2 with x_3, …). Instead each input index n must be paired with the sample whose index is n's 4-bit binary representation reversed — the direct hardware consequence of the recursive even/odd splitting in Section 1.1:

$$n = (n_3 n_2 n_1 n_0)_2 \;\longmapsto\; \text{reverse}(n) = (n_0 n_1 n_2 n_3)_2$$

The table below shows this mapping, which is exactly the pairing used to wire `x0..x15` into the eight Stage-1 butterfly instances (`u1_s1` through `u8_s1`):

n	bin(n)	reversed	pair	n	bin(n)	reversed	pair
0	0000	0000	0	8	1000	0001	1
1	0001	1000	8	9	1001	1001	9
2	0010	0100	4	10	1010	0101	5
3	0011	1100	12	11	1011	1101	13
4	0100	0010	2	12	1100	0011	3
5	0101	1010	10	13	1101	1011	11
6	0110	0110	6	14	1110	0111	7
7	0111	1110	14	15	1111	1111	15

Because inputs are real-valued samples, $a_i = b_i = 0$ (tied to `ZERO_16`) for every Stage-1 butterfly, and every Stage-1 twiddle is $W16\_0 = 1\angle 0°$ (a trivial multiply, since $H = 1$ pairs are always combined with unity gain at the first level of the recursion).

3.2 Twiddle Factor Set

Only 8 distinct complex twiddle values ($W16\_0$ through $W16\_7$, covering angles 0° through 157.5° in 22.5° steps) are stored as `localparam` constants, even though a full 16-point DFT nominally involves 16 twiddle angles. This is possible because of the symmetry

$$W_N^{k+N/2} = -\,W_N^k$$

the remaining 8 angles (180°–337.5°) are just the negatives of $W16\_0$–$W16\_7$, and that sign flip is already handled for free by the butterfly's $A - W \cdot B$ subtraction path. This halves the required twiddle constant storage.

Twiddle	θ	$\cos\theta$	$-\sin\theta$	Q1.14 Re	Q1.14 Im
$W16\_0$	0°	1.0000	0.0000	`0x4000`	`0x0000`
$W16\_1$	22.5°	0.9239	−0.3827	`0x3B21`	`0xE8C5`
$W16\_2$	45°	0.7071	−0.7071	`0x2D41`	`0xD2BF`
$W16\_3$	67.5°	0.3827	−0.9239	`0x187E`	`0xC4DF`
$W16\_4$	90°	0.0000	−1.0000	`0x0000`	`0xC000`
$W16\_5$	112.5°	−0.3827	−0.9239	`0xE782`	`0xC4DF`
$W16\_6$	135°	−0.7071	−0.7071	`0xD2BF`	`0xD2BF`
$W16\_7$	157.5°	−0.9239	−0.3827	`0xC4DF`	`0xE8C5`

3.3 Stage-by-Stage Interconnect

Each stage instantiates 8 butterflies that read from the previous stage's output wire array (`s1_r/s1_i`, `s2_r/s2_i`, `s3_r/s3_i`) and write into the next one, with the final stage driving the module's output ports directly. The butterfly distance H doubles each stage

$$H: \ 1 \rightarrow 2 \rightarrow 4 \rightarrow 8$$

and the number of distinct twiddle angles used per stage also doubles ($1 \rightarrow 2 \rightarrow 4 \rightarrow 8$), which is the standard radix-2 DIT signal-flow-graph pattern.

Stage 1 — Distance $H = 1$ (bit-reversed real inputs, twiddle $= W16\_0$ only)

Instance	Inputs (A, B)	Twiddle	Outputs
u1_s1	x0, x8	$W16\_0$	s1[0], s1[1]
u2_s1	x4, x12	$W16\_0$	s1[2], s1[3]
u3_s1	x2, x10	$W16\_0$	s1[4], s1[5]
u4_s1	x6, x14	$W16\_0$	s1[6], s1[7]
u5_s1	x1, x9	$W16\_0$	s1[8], s1[9]
u6_s1	x5, x13	$W16\_0$	s1[10], s1[11]
u7_s1	x3, x11	$W16\_0$	s1[12], s1[13]
u8_s1	x7, x15	$W16\_0$	s1[14], s1[15]

Stage 2 — Distance $H = 2$ (twiddles $= W16\_0$, $W16\_4$)

Instance	Inputs (A, B)	Twiddle	Outputs
u1_s2	s1[0], s1[2]	$W16\_0$	s2[0], s2[2]
u2_s2	s1[1], s1[3]	$W16\_4$	s2[1], s2[3]
u3_s2	s1[4], s1[6]	$W16\_0$	s2[4], s2[6]
u4_s2	s1[5], s1[7]	$W16\_4$	s2[5], s2[7]
u5_s2	s1[8], s1[10]	$W16\_0$	s2[8], s2[10]
u6_s2	s1[9], s1[11]	$W16\_4$	s2[9], s2[11]
u7_s2	s1[12], s1[14]	$W16\_0$	s2[12], s2[14]
u8_s2	s1[13], s1[15]	$W16\_4$	s2[13], s2[15]

Stage 3 — Distance $H = 4$ (twiddles = $W16\_0, W16\_2, W16\_4, W16\_6$)

Instance	Inputs (A, B)	Twiddle	Outputs
u1_s3	s2[0], s2[4]	$W16\_0$	s3[0], s3[4]
u2_s3	s2[1], s2[5]	$W16\_2$	s3[1], s3[5]
u3_s3	s2[2], s2[6]	$W16\_4$	s3[2], s3[6]
u4_s3	s2[3], s2[7]	$W16\_6$	s3[3], s3[7]
u5_s3	s2[8], s2[12]	$W16\_0$	s3[8], s3[12]
u6_s3	s2[9], s2[13]	$W16\_2$	s3[9], s3[13]
u7_s3	s2[10], s2[14]	$W16\_4$	s3[10], s3[14]
u8_s3	s2[11], s2[15]	$W16\_6$	s3[11], s3[15]

Stage 4 — Distance $H = 8$ (twiddles = $W16\_0 \ldots W16\_7$, outputs = natural-order $X_0..X_{15}$)

Instance	Inputs (A, B)	Twiddle	Outputs
u1_s4	s3[0], s3[8]	$W16\_0$	X0, X8
u2_s4	s3[1], s3[9]	$W16\_1$	X1, X9
u3_s4	s3[2], s3[10]	$W16\_2$	X2, X10
u4_s4	s3[3], s3[11]	$W16\_3$	X3, X11
u5_s4	s3[4], s3[12]	$W16\_4$	X4, X12
u6_s4	s3[5], s3[13]	$W16\_5$	X5, X13
u7_s4	s3[6], s3[14]	$W16\_6$	X6, X14
u8_s4	s3[7], s3[15]	$W16\_7$	X7, X15

3.4 Why It Is Connected This Way

- **In-place, no separate memory:** because a radix-2 DIT FFT can be computed "in place" (the two outputs of a butterfly overwrite the same two positions its inputs came from in the signal-flow graph), the design simply chains four flat arrays of wires (`s1..s3`, plus the final output ports) instead of needing RAM, an address generator, or a bit-reversal memory permutation circuit. The reordering is done once, for free, at the Stage-1 port wiring (Section 3.1) purely by choosing which module output signal is called which array index.

- **Fully unrolled, feed-forward pipeline:** all 32 butterflies exist simultaneously in hardware (rather than one physical butterfly being time-multiplexed across 32 operations), and each stage's registers (Section 2.2) feed directly into the next stage's combinational inputs. This gives the whole FFT a fixed latency of 4 clock cycles, and a new complete 16-point transform can be accepted every clock cycle once the pipeline is full — i.e. the design has a throughput of 1 FFT/cycle at the cost of the area of 32 butterfly units, a classic area-for-throughput trade typical of streaming DSP front-ends.

- **Twiddle selection follows the signal-flow graph directly:** the `wr`/`wi` ports of every instance are wired to whichever `W16_x` `localparam` the Cooley–Tukey decomposition calls for at that node (Sections 3.2–3.3), so no runtime twiddle-address computation or ROM lookup logic is needed — the angle assignment is fixed by the netlist itself.

- **Common `clk`/`rst_n`/`enable` fan-out:** every one of the 32 butterfly instances shares the top module's `clk`, `rst_n` and `enable` ports, so the whole pipeline resets and advances syn-

chronously as one unit; asserting `enable` for one cycle pushes exactly one sample set through that pipeline stage, keeping every stage's data precisely aligned with its neighbours.

- **Natural-order output for free:** because Stage 1 undoes the bit-reversal on the way in, Stage 4's outputs already land on the correctly-numbered `X0_r..X15_r` / `X0_i..X15_i` ports with no output-side reordering network required.

4 Summary

The design realizes a 16-point radix-2 Decimation-in-Time FFT as a fully-pipelined, fully-unrolled datapath. Section 1 established why the Cooley–Tukey decomposition ($O(N \log_2 N)$ vs. $O(N^2)$) motivates breaking the transform into 4 stages of 8 butterflies. Section 2 detailed the `butterfly.v` module's implementation of the core

$$A' = A + W \cdot B, \qquad B' = A - W \cdot B$$

equations in Q1.14 fixed-point arithmetic with careful bit-growth handling and one-cycle output pipelining. Section 3 showed how `top_fft_16.v` wires 32 such butterflies into 4 chained stages, using the classic bit-reversed-input / natural-order-output property of DIT FFTs to avoid any explicit reordering hardware, and using only 8 stored twiddle constants thanks to $W_N^{k+N/2} = -W_N^k$ symmetry — together producing a streaming FFT core with 4-cycle latency and single-cycle throughput.